

The Agentic Operating Model

Aligning People, Process, and Technology for Production AI at Scale

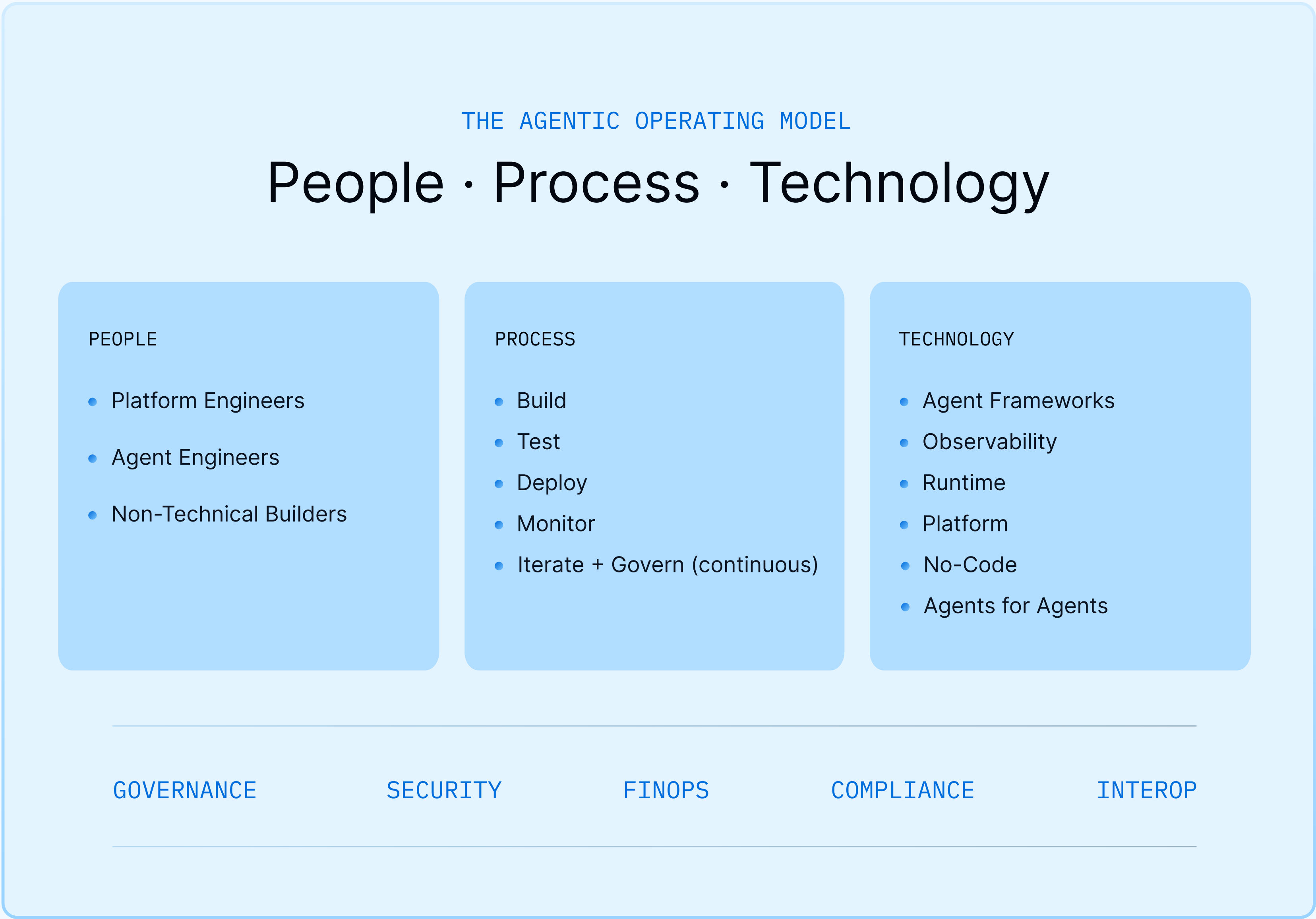


Table of Contents

0.0	Foreword	3
1.0	Part I · People	5
1.1	1. The Three Rings: Who Builds, Who Adopts	5
1.2	2. The Three Disciplines: Hire and Reskill	8
2.0	Part II · Process	10
2.1	3. The Agent Development Lifecycle	10
2.2	4. Test in Depth	12
2.3	5. Deploy, Monitor, and Govern	14
3.0	Part III · Technology	15
3.1	6. The LangChain Stack Supports Every Phase	15
4.0	Part IV · Cross-cutting Concerns	19
4.1	7. The Agentic Risk Taxonomy	20
4.2	8. FinOps for Agentic Workloads	21
4.3	9. Standards Alignment	22
4.4	10. Independence Across People, Process, and Technology	22
5.0	Part V · Journey	23
5.1	11. The Maturity Journey	23
5.2	12. In Practice	26
5.3	13. Critical Success Factors	27
	Conclusion	28

Foreword

Agents are moving from prototypes to production today. They handle customer interactions, automate workflows, and drive measurable business outcomes.

But the teams building the best agents are not following the traditional software development lifecycle. Unlike traditional software, agents are non-deterministic, stateful, and dependent on models, prompts, and tools that change without warning. The engineering playbooks enterprises have relied on to build and operate software for decades do not translate.

The pace of change in AI also requires a different approach and mindset across the organization, both bottom-up and top-down.

The traditional development playbook does not fit

A well-staffed team with no agent improvement process ships agents that break silently. A rigorous process with inadequate tooling creates manual overhead that stops scaling. Powerful technology in poorly structured teams produces fragmented, ungoverned systems.

Traditional software is deterministic. Unit tests validate logic, CI/CD enforces gates, monitoring alerts on exceptions that should not happen. Agents break every one of these assumptions:

DIMENSION	TRADITIONAL SOFTWARE	AGENTIC SYSTEMS
Behavior	Deterministic, code-driven	Probabilistic, prompt-driven
Testing	Unit tests validate logic	Evaluation suites measure quality
Debugging	Stack traces find the root cause	Trace analysis inspects the reasoning
Versioning	Code commits	Prompts, models, data, tools, and code
Monitoring	Exceptions, error rates	Accuracy, hallucination rates, confidence drift, cost per interaction
Iteration	Specs to code changes	Failure clusters to prompt, model, or tool refinement
Failure mode	Crash, exception, wrong output	Plausible-but-wrong output, goal drift, tool misuse, cost overrun
Builder population	Software engineers	Engineers and non-engineers (SMEs, ops, marketing, support, finance, HR)

The last row in the table above is the one most enterprises underweight. ML used to be the province of a narrow elite of ML engineers and data labelers. Agents have widened the aperture: anyone who can write a prompt, define an evaluation, or wire a tool can meaningfully shape an agent. AI fluency, not engineering headcount, is the new bottleneck.

The Agentic Operating Model

We recently introduced the Agent Development Lifecycle (ADLC), a continual product development process for building reliable agents. This whitepaper introduces The Agentic Operating Model (AOM): a single operating model that aligns People, Process, and Technology so an enterprise can build, test, deploy, monitor, govern, and continuously improve agent systems at scale.

The Agentic Operating Model is the scaffolding that lets the LangSmith-enabled ADLC run reliably inside an enterprise. Because the models, tools, and frameworks beneath every agent change constantly, neutrality and vendor independence are a design requirement rather than an afterthought.

And most importantly, agents produce vast amounts of behavioral data, with the improvement opportunities buried inside that data. Manually reviewing agent behavior is not feasible, so accelerating agent delivery is itself contingent on using agents.

Three principles shape the AOM:

PRINCIPLE	WHAT IT MEANS
Agent engineering is a new discipline	Agent behavior is not captured in code; it lives in the traces of its execution. Prompt engineering, evaluation, and observability are engineering priorities from day one. Treating them as polish that comes after the agent ships is the most expensive mistake enterprises make.
Establish enabling constraints	Allow teams to work fast without breaking things. This starts with establishing enterprise consistency for agent infrastructure. Speed and domain relevance need decentralized teams, and increasingly, decentralized non-engineer builders. You need all three.
Improvement must be continuous	The organizations that compound advantage in AI are the ones that close the loop fastest, from a production failure to a measurable quality improvement.

Three vertical pillars (People, Process, Technology) intersect with the cross-cutting concerns (Governance, Security, FinOps, Compliance, Interoperability). Each pillar is necessary; the model works only when all are aligned.

What this document covers

PART	WHAT IT COVERS
I · People	The three rings of builders and the skills each ring brings
II · Process	The ADLC, and how each skill maps to each phase
III · Technology	The agent engineering stack that supports the process, end to end
IV · Cross-cutting concerns	How the technology enforces what the people and process commit to across Governance, Security, FinOps, Compliance, and Interoperability
V · Journey	How an organization matures through these pillars, and how LangChain accelerates each transition

Part I · People

1. The Three Rings: Who Builds, Who Adopts

The most common organizational failure in enterprise AI programs is going to an extreme.

- Too centralized: one team builds all agents and delivery becomes a bottleneck.
- Too decentralized: every team picks their own stack, and the result is sprawl, duplicated infrastructure, and ungoverned risk.
- Too engineer-only: the people closest to the work (operators, analysts, SMEs) never participate in agent creation, and the organization’s institutional knowledge stays trapped outside the loop.

The Agentic Operating Model resolves all three by organizing builders into three concentric rings, with governance distributed across them rather than parked in a separate review board.



RING	WHO	WHAT THEY OWN	HOW THEY PARTICIPATE
Ring 1 · Platform Engineers	Horizontal platform / applied-AI / platform-governance	Model gateway, tool gateway, evaluation infrastructure, observability backplane, audit logging, policy engine, reference patterns	They build the substrate, operate the eval flywheel for the portfolio, set the governance bar, curate the data, and run evaluations that govern quality.
Ring 2 · Agent Engineers	Domain engineering teams (product, ops, customer, finance, etc.)	Domain-specific agents, risk classification at design time, evaluation thresholds for their domain, compliance ownership for the agents they ship	Self-serve the platform to build and ship domain agents within paved-path patterns and governance bar. Co-building with the Platform team is an optional accelerator for first use cases, not a prerequisite.
Ring 3 · Non-Technical Builders	SMEs, operators, analysts, support, marketing, sales, finance, HR. Anyone who works with the domain knowledge an agent needs.	Prompts, skills, evaluation criteria, ground truth, feedback on production behavior, and increasingly, their own agents	Use agents in their daily work. Build agents in no-code surfaces.

Governance, security and compliance are a set of responsibilities baked into all three rings, with shared infrastructure provided by Ring 1 and shared accountability owned by Rings 2 and 3.

Start small and scale

Not all organizations and teams require clear distinctions between platform and agent engineering teams at the beginning of their agent engineering journey. In this world, it helps to collapse Rings 1 and 2 into a single ring, while Ring 3 remains distinct.

As agent adoption increases across teams, an organic distinction between Rings 1 and 2 will emerge to maintain agility and scale.

For the remainder of this document, merge Rings 1 and 2 if you are early in your agent journey.



Why Ring 3 is the underweighted lever

Most enterprise AI strategies have a serious Ring 1 plan and a workable Ring 2 plan. Ring 3 is where they fall short, and it is also where the largest fraction of the workforce sits.

In classical ML, the answer to "who builds the model?" was a small team of specialists. With agents, the answer changes. The people who know what good looks like in the domain are no longer separate from the people who can shape the agent. A claims adjuster who has handled a thousand claims can write the prompt that handles the next thousand better than a prompt engineer who has handled none. A support lead who has triaged ten thousand tickets can define the eval that matters more accurately than a data scientist who is two steps removed.

No-code surfaces like LangSmith Fleet (LangChain's natural-language agent builder, covered in Part III) make it easy to create prototypes and internal agents, often for productivity and collaboration. A non-engineer describes the agent's behavior in natural language, and the system produces a working agent that runs on the same runtime, observability, and governance backbone as a code-built one.

This changes what an enterprise must invest in:

INVESTMENT	WHY IT MATTERS
Universal AI fluency	Every employee should be able to use agents in their daily work. The base rate of “AI-aware work” determines how fast the organization compounds. Change management and demystifying AI agents need to be a conscious and coordinated effort.
AI literacy for builders in Ring 3	A growing share of agents should be built by the SMEs who own the domain. Train them on prompt design, evaluation thinking, and where the governance guardrails are.
Tooling that meets builders where they are	If the only way to build is to write Python, Ring 3 is excluded by construction. No-code tooling, with shared governance underneath, are the entry point.
Role modeling and cultural permission	Many enterprises still treat “building software” as someone else’s job. Leaders have to make it explicit that agent building is part of every role’s surface area, the same way spreadsheet building once was.

AI fluency is the new spreadsheet. Treating it as a specialist skill is the most predictable way to be overtaken by an organization that treats it as a baseline.

The Agent Engineering Hub, not a gatekeeping committee

The three-ring pattern operates as an Agent Engineering Hub. When present, Ring 1 produces patterns, primitives, and reference implementations, then spreads them outward through Rings 2 and 3. It should not be a review board that approves or denies use cases. Gatekeeping is top-down and slow, while a hub is collaborative and multiplies enablement. Think of it as a train-the-trainer pattern adapted for agent engineering, extended one ring further to include the people who do the work the agents are automating.

Governance, security and compliance

Three functions share governance accountability. They are usually not three separate teams.

FUNCTION	WHO TYPICALLY OWNS IT
Day-to-day operation	Platform team (Ring 1)
Oversight and audit	Security / InfoSec
Mandate and policy	Compliance

Existing security and compliance functions do not need to be replaced. They need AI-specific awareness around new threat vectors: prompt injection, token leakage through chat surfaces, context-window data exfiltration. Part IV returns to how governance is enforced through the technology pillar.

2. The Three Disciplines: Hire and Reskill

Building an agent-capable organization does not require replacing existing talent. It requires hiring a small number of specialists to anchor new capabilities, and systematically reskilling high-performing employees in adjacent disciplines.

Agent engineering is not one new role. It is three existing disciplines, each expanded. This framing, drawn from [LangChain's overview of agent engineering as a new discipline](#), clarifies who you actually hire, who you reskill, and how those people fit into the ADLC in Part II.

Discipline 1 · Product Thinking

Defining what the agent should do, writing the prompts that drive its behavior (often thousands of lines), and writing the evaluations that say whether it is actually working. Think of this as product management with a wider surface area, where behavior is specified in the artifact that drives the behavior rather than in a separate spec document.

WHERE IT LIVES	RESKILL FROM	NEW SKILLS TO LEARN
Rings 1, 2, and 3 (Ring 3 most underrated)	Product managers, business analysts, SMEs, ops leads, content/comms specialists	Prompt design, eval criteria definition, failure-cluster analysis, iteration loops over specs

Discipline 2 · Engineering

Building the tools the agent calls, the runtime that executes it durably, the human-in-the-loop pauses that keep it safe, and the platform primitives the rest of the organization builds on. Think of this as software engineering with a wider surface area, covering state, memory, tool orchestration, non-deterministic debugging, and sandboxing. Responsibility for implementing agents' performance improvements is owned primarily by engineers.

WHERE IT LIVES	RESKILL FROM	NEW SKILLS TO LEARN
Rings 1 and 2	Backend / full-stack engineers, DevOps / infra engineers, ML engineers	Tool calling patterns, state and memory, durable execution, model serving, sandbox isolation, prompt-as-artifact discipline

Discipline 3 · Data Science

Building the systems that measure agent performance: evaluation harnesses, A/B testing, monitoring, and error analysis on production traces. Think of this as data science applied to reliability metrics for unpredictable systems, not standard statistical analysis on tidy datasets. This skillset is often in high demand, and in a world of agents that emit millions of traces it requires dedicated focus. Continuous-improvement agents that quantify and iterate on other agents' performance are a key force-multiplier for these teams.

WHERE IT LIVES	RESKILL FROM	NEW SKILLS TO LEARN
Rings 1 and 2	Data scientists, QA engineers, ML engineers	LLM-as-judge design, golden / synthetic / adversarial dataset construction, regression suites, online vs offline eval, cost-per-correct-answer thinking

What to hire vs reskill, by discipline

DISCIPLINE	HIRE TO ANCHOR (TYPICAL 1–3 FTE)	RESKILL TO SCALE
Product Thinking	One senior Agent PM with eval and prompt fluency	Existing PMs, BAs, and SMEs in every BU
Engineering	One agent platform engineer, one applied-AI engineer	Existing backend, DevOps, and ML engineers
Data Science	One evaluation engineer with red-team experience	Existing data scientists and QA engineers
Adjacent specialists	AI security engineer, conversational/trust designer (often one of each at the org level)	Pull from security and UX

Adoption is the multiplier

The three disciplines describe who builds, while adoption fluency tells you whether the rest of the org gets value from what gets built. An enterprise that hires brilliant agent engineers and never trains the rest of the workforce has built a Formula 1 car and forgotten to teach anyone to drive. Programs that work tend to share a few features:

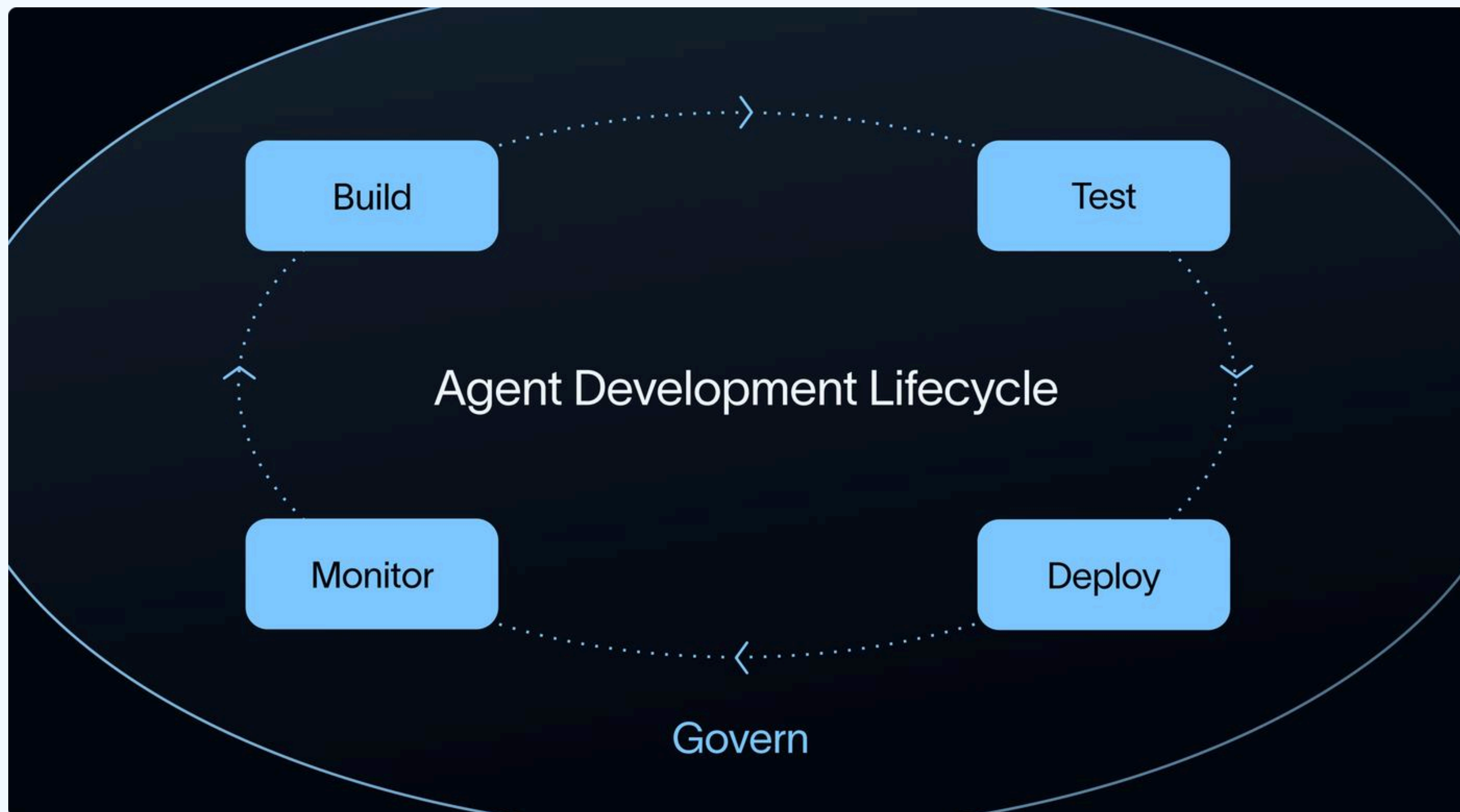
- Mandatory AI literacy training for every employee, with role-specific tracks.
- Internal communities of practice where Ring 3 builders share working agents and ask Ring 2 teams for help.
- Recognition and incentives that reward agent building, not just agent consumption.
- A no-code-first policy for new automations under a defined risk threshold, with a paved path to upgrade into Ring 2 if the agent grows.

The three disciplines from this section now map directly onto the Agent Development Lifecycle in Part II. Each phase of the ADLC has a primary discipline and a primary ring. The Agentic Operating Model only works when the right combination shows up at the right phase.

Part II · Process

3. The Agent Development Lifecycle

Traditional software follows a predictable arc: define requirements, build, test, ship. The Agent Development Lifecycle (ADLC) is different. The cycle is shorter, continuous, and driven by evaluation rather than specs.



The lifecycle is four phases plus two continuous practices. The phases are sequenced for any single agent release. The continuous practices run across all phases all the time.

Skills × phases: who shows up where

Every phase requires a primary discipline (Section 2) and a primary ring (Section 1). The Agentic Operating Model fails when a phase runs without one of these.

PHASE	PRIMARY DISCIPLINE	PRIMARY RING	WHAT GOOD LOOKS LIKE
Build	Product Thinking + Engineering	Ring 2 builds; Ring 1 provides patterns; Ring 3 builds in Fleet	Working agent with externalized prompts and typed tool schemas
Test	Data Science + Product Thinking	Ring 2 owns; Ring 1 provides infra; Ring 3 contributes ground truth	Pass/fail signal against quality thresholds before promotion
Deploy	Engineering	Ring 1 provides the runtime; Ring 2 operates the rollout	Agent live with version control, HITL, and rollback
Monitor	Data Science + Engineering	Ring 1 provides observability; Ring 2 watches; Ring 3 reports issues	Visibility into actual behavior and where it fails
Iterate (continuous)	All three disciplines	All three rings	Failure → root cause → fix → re-evaluate, in days not quarters
Govern (continuous)	All three disciplines	All three rings, plus Security and Compliance	Cost, access, discoverability under control as the portfolio scales

Iteration and governance often constitute more than 80% of agent funding, so a conscious and deliberate approach to automation is non-negotiable.

Build: choosing the right abstraction

The Build phase opens with a choice of abstraction level. Get this wrong and the agent is harder to maintain than it should be. Get it right and the team moves at a pace that matches the cadence the rest of the framework demands.

ABSTRACTION	BEST FOR	PRIMARY RING	WHAT IT PROVIDES
Frameworks (e.g., <u>LangChain</u>)	Composing model calls, tools, prompts, retrieval, structured outputs	Ring 2	The building blocks. Vendor-neutral integrations across the ecosystem
Runtimes (e.g., <u>LangGraph</u>)	Stateful, multi-step agents with control flow, durability, human intervention, branching, looping, pausing, resuming	Ring 2	The execution substrate. Graph-based state management with checkpointing
Harnesses (e.g., <u>Deep Agents</u>)	Long-horizon autonomous workloads with skills, MCP integrations, hooks, middleware, filesystem support	Ring 1 or advanced Ring 2	The opinionated scaffolding. Higher-level patterns for autonomous agents
No-code (e.g., <u>LangSmith Fleet</u>)	Non-engineer participation. SMEs and operators building their own agents	Ring 3	The natural-language entry point. Same runtime and governance as code-built agents

4. Test in Depth

The Test phase is the engine of agent improvement. Evaluation is the most universally weak area in enterprise AI programs, and treating it as a continuous practice rather than a deployment checkpoint is the single biggest differentiator between organizations that compound on AI investment and those that stall.

Dataset construction (Data Science + Ring 3 SMEs)

Datasets are the ground truth for agent quality. Ring 3 SMEs are usually the most valuable contributors, while the Data Science discipline owns the framework.

DATASET TYPE	PURPOSE	HOW TO BUILD
Golden / Curated	Validate against real-world ground truth	SME curation plus production trace selection
Synthetic	Comprehensive edge-case coverage	LLM-generated test cases with SME review
Trajectory	Verify execution paths through the graph	Capture expected tool sequences and node traversal
Regression	Prevent quality degradation	Curate from previously resolved failures
Adversarial	Test injection and jailbreak resistance	Red-team contributions, public threat datasets

Evaluator selection

Evaluators score agent runs against the datasets.

EVALUATOR TYPE	BEST FOR	TRADEOFF
Ground-truth correctness	Tasks with a knowable right answer	Low
Criteria-based (LLM-as-judge)	Grounding, policy adherence, efficiency, helpfulness	High, use selectively
Deterministic / heuristic	Schema, format, exact match	Low, use for all verifiable assertions
Human annotation queues	Nuanced quality judgment, dataset curation	High, but generates ground truth
Pairwise comparison	Ranking prompt variants, A/B decisions	Medium
Align evals	Aligning LLM-as-judge evaluators with human judgment; reducing scorer drift	Medium upfront (needs human-labeled examples), low ongoing

Experiments, simulations, and the iteration cycle

Beyond static datasets, the Test phase includes experiments (prompt variants, models, retrieval strategies, tool schemas compared side by side) and simulations (multi-turn evals for conversational, voice, and support agents).



ROOT CAUSE	FIX STRATEGY
Prompt not following instructions	Add examples, clarify constraints, few-shot demonstrations
Model capability limitation	Upgrade model for that specific node only
Routing or classification error	Enhance supervisor prompt with routing rules and examples
Knowledge gap	Expand knowledge base, refine retrieval strategy
Tool misuse	Tighten tool schemas, improve tool descriptions, add guardrails

This cycle is what LangSmith Engine (Section 6) automates. Engine detects recurring failures in traces, diagnoses the root cause, proposes a fix, and deploys an evaluator to catch regressions, closing the loop the Data Science discipline runs by hand at lower maturity stages.

Closing the Loop: From Observation to Action

Observability alone does not improve agents. The value of traces, evals, and dashboards is only realized when teams can move quickly from signal to diagnosis to fix, and do so repeatedly, at the pace production demands. In practice, this loop breaks down not because teams lack data, but because the work of interpreting that data is slow, manual, and competes with the work of building.

The teams shipping the best agents have found ways to make this iteration cycle a first-class discipline, not an afterthought triggered by customer complaints. They treat production failures as structured inputs to a continuous improvement process: clustering them by pattern, tracing them to root cause, proposing a targeted fix, and verifying the fix holds before it reaches users again. The faster and more reliably a team can run this loop, the better their agents become.

As agent complexity and trace volume grow, running this loop by hand becomes the bottleneck. The natural next step is to ask: what parts of this cycle can be systematized, accelerated, or automated entirely, and what would that unlock for the teams responsible for agent quality?

5. Deploy, Monitor, and Govern

Deploy (Engineering · Ring 1 + Ring 2)

The Deploy phase is about reliable execution environments, not just shipping code.

- Durable runtime with checkpointed state. Long-running agents survive infrastructure failures and resume from the last checkpoint.
- Human-in-the-loop checkpoints for sensitive operations. The agent pauses, a human reviews and approves, the agent continues.
- Sandboxes for code writing and filesystem access. Isolated execution that cannot escape its boundary.
- Context Hub for the behavior files that shape an agent (prompts, skills, policies, examples). Versioned, environment-tagged, managed independently from agent code.
- Gradual rollout with canary at 5–10%, then 25%, 50%, 75%, then full, each stage monitored before advancing.

Monitor (Data Science + Engineering · Ring 1 + Ring 2 + Ring 3 feedback)

When a traditional application fails, you read a stack trace. When an agent fails, you need the reasoning chain. That requires four kinds of production visibility:

PILLAR	WHAT IT CAPTURES
Traces	Full trajectory of an agent run: inputs, model calls, tool invocations, outputs, final response
Signals	Automated quality scoring: LLM-as-judge, regex patterns, policy compliance
Feedback	Explicit (ratings, corrections) and implicit (escalation, abandonment) attached back to the run. Ring 3 is usually the largest source.
Dashboards	Aggregate views of usage, latency, cost, tool calls, evaluator scores, and failure patterns

Standard application metrics (latency, uptime, error rate) miss the agent-specific failures: the agent calling the wrong tool, relying on the wrong context, skipping a required approval step, or producing an answer that sounds plausible but is wrong. These failures are invisible without agent-specific traces and signals.

Govern (Continuous · all rings)

Governance is a continuous practice that operates across Build, Test, Deploy, and Monitor at the same time. Three concerns matter most as the agent portfolio scales. Each maps to a specific technology enforcement point in Part III.

GOVERN CONCERN	WHAT GOOD LOOKS LIKE	ENFORCEMENT POINT
Cost	Token budgets per agent, per tenant, per time window. Spend alerts. Cost attribution per agent, per node, per feature.	<u>LangSmith LLM Gateway</u> (Section 6)
Tool access	Every tool call passes through a policy engine. Scoped tokens, audit trails, HITL for sensitive operations.	Tool Gateway + LangGraph interrupts
Discoverability	Prompts, skills, tools, and agents are reusable across teams. A new team starting on a new agent should not be rebuilding what already exists.	<u>SmithDB</u> + LangSmith prompt/skill registry

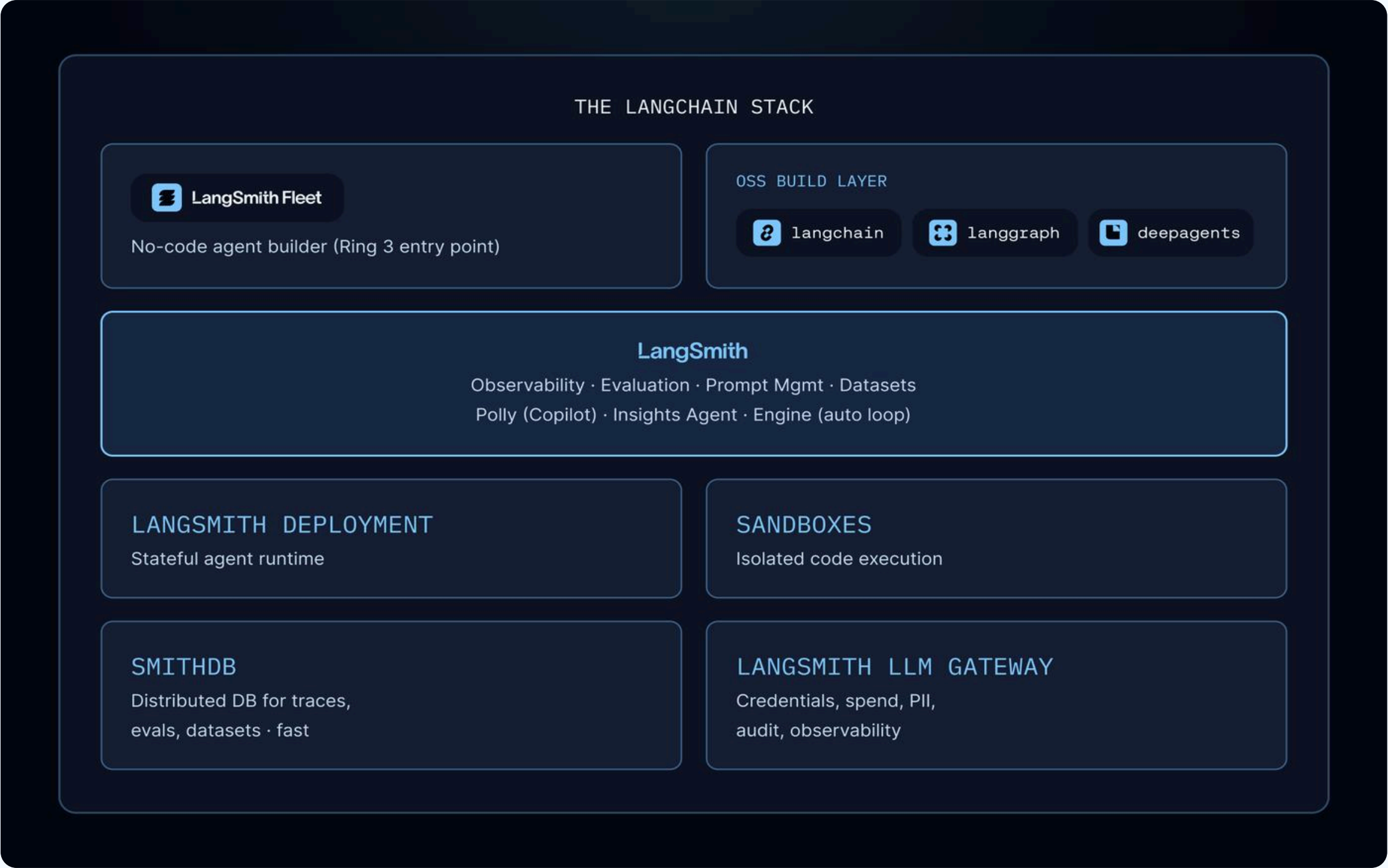
This is where the People → Process story hands off cleanly to Technology: every governance concern named here is enforced by a specific component of the LangChain stack in Part III.

Part III · Technology

6. The LangChain Stack Supports Every Phase

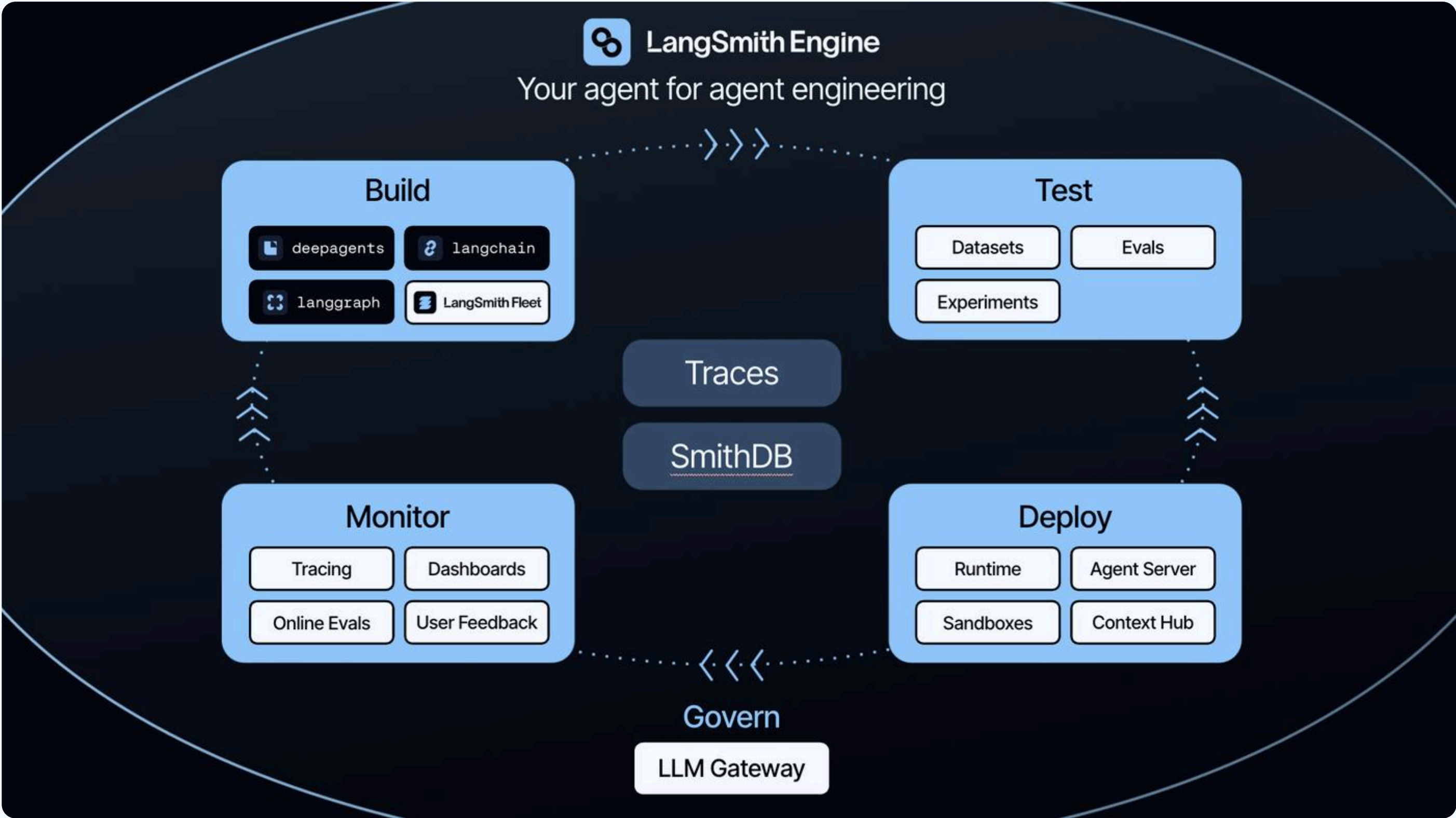
The Agentic Operating Model is enabled by an integrated stack covering every phase of the ADLC and every governance concern from Part II.

Unlike point solutions that solve one problem in isolation, the components are designed so each strengthens the others.



The mapping from process to stack is direct:

ADLC PHASE	PRIMARY STACK COMPONENTS
Build	Deep Agents, LangChain, LangGraph (code) · Fleet (no-code)
Test	Datasets, Evals, Annotation Queues, Simulations
Deploy	Runtime, Agent Server, Sandboxes, Context Hub
Monitor	Tracing, Dashboards, Online Evals, User Feedback
Iterate	LangSmith Engine (auto loop), Chat (assisted), Insights
SmithDB (data layer)	Traces, SmithDB
Govern (continuous)	LangSmith LLM Gateway, Tool Gateway, Policy engine, SmithDB audit



Build: framework, runtime, harness, no-code

The four abstractions introduced in Section 3 are concrete products in the stack:

Deep Agents (harness) adds structured planning, persistent memory, MCP integration, hooks, and middleware. Use it for long-horizon autonomous workloads.

LangChain (framework) composes model calls, tools, prompts, retrieval, and structured outputs. 600+ integrations across providers, vector stores, and tools. Use it for RAG pipelines, integration glue, and as the composition layer inside more complex agents.

LangGraph (runtime) provides state, control flow, durability, human intervention, branching, looping, pausing, resuming. Use it when the agent needs more than a linear chain.

LangSmith Fleet (no-code) is the Ring 3 entry point. Non-engineers describe the agent's behavior in natural language and the system produces a working agent. It runs on the same runtime, observability, governance, and trace history as code-built agents, so a Fleet agent can be picked up and extended in LangGraph without losing context.

Test: evaluation and datasets

LangSmith is where the inner loop of the ADLC lives. For the Test phase it provides Evaluation (offline + online, LLM-as-judge, deterministic validators, human annotation queues), Prompt Management (versioned artifacts with audit trail and rollback), and Datasets (manual curation, production sampling, synthetic generation). Test is covered in depth in Section 4; here it is the evaluation substrate the rest of the stack builds on.

Deploy: LangSmith Deployment and Sandboxes

LangSmith Deployment is the production runtime for stateful agents: durable execution, checkpointed state, horizontal scaling, thread management, versioned deployments, automatic rollback. Observability, evaluation, and deployment unified in a single platform.

LangSmith Sandboxes provide isolated execution environments for the riskier parts of agent work: code generation and execution, filesystem access, untrusted tool calls. A sandbox cannot escape its boundary, which is what makes Tier 2 and Tier 3 agents (the risk tiers defined in Section 7) deployable at all without taking on unbounded blast radius.

Monitor: observability

Once an agent is live, LangSmith Observability traces every run automatically with full state and cost. Observability alone is not enough. Most enterprises arrive at LangSmith for observability, but observability is just the start of agent improvement. The harder problem appears once traces are flowing: trace volume compounds, and each trace is heterogeneous and deeply nested. Left alone, that volume and complexity compounds into a source of paralysis rather than analysis.

Iterate: actionable observability

The iteration layer is how LangChain delivers actionable observability, actively converting traces into signal rather than noise. Three components do that work on top of the LangSmith substrate:

- Chat: an interactive AI copilot for trace analysis and debugging, used by the Data Science and Engineering disciplines during failure investigation, serving as both a peer and an aid.
- Insights: automated pattern discovery that clusters agent behavior, surfaces where the agent performs well versus poorly, and most importantly, shows behavioral drift over time. It is instrumental in uncovering emerging user behavior or the impact of changing compliance controls. Agent teams can compress the analysis phase and directly leverage these insights into their sprint planning rituals.
- LangSmith Engine: is the agent that runs the iteration cycle on your behalf. Where a human spends hours sifting through traces to find failures, Engine automatically clusters production issues by severity and root cause, surfacing 50 or more actionable issues per month within hours of them occurring, not after a customer complaint.

Once issues are identified, Engine doesn't stop at diagnosis. It proposes targeted fixes (a prompt edit, a Context Hub update, or a code PR), each grounded directly in the traces that revealed the problem. It simultaneously generates the evaluators and dataset examples needed to prevent that class of failure from returning, turning every production incident into a permanent improvement to your eval suite. Engine is what lets the Data Science discipline scale beyond what a human team can review by hand. It is the Agentic Operating Model's "Stage 4 / Scaling" capability turned into a product.

The compounding effect is significant. Teams operating Engine ship agent improvements weekly rather than monthly, catch regressions before they reach users, and redirect engineering time from reactive debugging toward building new capabilities. At scale, Engine replaces what would otherwise require 2–3 dedicated engineers working full-time, at roughly one-tenth the cost.

Together they show up throughout your journey.

Solving the cold start: Insights gives a team its first plain-English read on how an agent is actually being used, and Engine provides the practical substrate to start running the ADLC in earnest.

Sustaining and scaling: strategic use of Insights reports keeps Rings 2 and especially 3 engaged with production reality, while Engine stretches the limited capacity of Ring 1 and Ring 2 much further.

All of this continual learning requires complex operations over traces at scale, which is why SmithDB sits underneath it. It is LangChain's bet that long-term continual learning, agents improving agents, demands purpose-built infrastructure, so these tools stay durable as the trace corpus grows.

SmithDB: the data layer

SmithDB is the purpose-built distributed database that powers LangSmith's observability, and serves both human and agent consumers. Engineered in Rust on Apache DataFusion and Vortex, it is designed for the shape of agent telemetry that general-purpose databases handle slowly and expensively: deeply nested traces with hundreds of spans, multi-modal payloads, and long-open spans that arrive in pieces over hours.

The practical effect: slow observability is itself a bottleneck in the agent development loop. SmithDB makes the queries that loop depends on fast enough to use continuously, instead of as after-the-fact reviews.

CAPABILITY	WHY IT MATTERS IN THE ADLC
Trace-tree loading (P50 ~92ms)	Add examples, clarify constraints, few-shot demonstrations
Full-text search across payloads (P50 ~400ms)	Upgrade model for that specific node only
Metadata and feedback filtering	Enhance supervisor prompt with routing rules and examples
JSON queries on structured outputs	Expand knowledge base, refine retrieval strategy
Thread reconstruction	Tighten tool schemas, improve tool descriptions, add guardrails
Cost, latency, and token aggregations	The data behind the FinOps attribution tables in Section 8

SmithDB powers high-volume organizations like Clay, Vanta, Unify, and Cogent Security.

Beyond the speed, SmithDB is where the artifacts of the Agentic Operating Model accumulate over time: every trace, evaluation, annotation, and dataset. By collecting, evaluating, and annotating traces today, an enterprise is curating the data that will shape future agents. Explicit examples of what "good" and "bad" look like will always matter in AI and ML system design.

Govern: LangSmith LLM Gateway

The LangSmith LLM Gateway is the centralized enforcement point for everything that crosses an LLM boundary. Rather than distributing API keys, spend rules, and PII controls across every agent and every team, the Gateway concentrates them at the model boundary.

GATEWAY CAPABILITY	WHAT IT ENFORCES
Credential management	API keys stored once in LangSmith; agents authenticate with LangSmith credentials, not provider keys
Spend limits	Hard limits at organization, workspace, API key, or user level; 402 hard-block on overrun
PII detection and redaction	Names, places, nationality, religion, political affiliation, ages, all redacted before leaving the gateway
Secrets redaction	Phone numbers, SSNs, API keys, tokens across major platforms
Audit logging	Administrative changes and every invocation logged
Real-time cost visibility	By workspace, user, and API key
Trace unification	Every Gateway request appears as a trace in LangSmith, so observability does not fragment across systems
Engine integration	Policy violations surface as issues in LangSmith Engine, with click-through to the originating trace
Multi-provider	Anthropic, OpenAI, Google Gemini today, with model-neutral architecture

The Gateway is where the governance commitments from Sections 1–5 become enforced in practice. Sections 7 and 8 return to it as the anchor for both governance and FinOps.

Part IV · Cross-cutting Concerns

Governance, security, FinOps, compliance, and interoperability are not a separate workstream. They are how the People commitments (who owns what) and the Process commitments (what gets checked when) become enforceable through the Technology stack.

This part covers them through four lenses: a risk taxonomy that scales controls to each agent's blast radius (Section 7), FinOps for token-based, multi-model workloads (Section 8), standards alignment for the regulations you must already answer to (Section 9), and vendor independence that keeps interoperability open across all three pillars (Section 10).

Security and compliance do not get standalone sections because they run through all four, enforced at the LLM Gateway, the risk tiers, and the audit trail rather than bolted on separately.

7. The Agentic Risk Taxonomy

As agents move from read-only retrieval to taking actions, governance becomes as critical as agent logic. The Agentic Operating Model classifies every agent into one of three tiers, with controls scaled to the tier. Classification is a Product Thinking + Engineering activity that happens at design time during Build, and is re-evaluated whenever tool or model authorization changes. Each tier inherits the controls of the one below it.

TIER	EXAMPLES	REQUIRED CONTROLS (ADDITIVE)
1: Bounded (read-only, informational)	Retrieval, summarization, classification, knowledge-base Q&A, content generation for human review	Input/output filtering (LLM Gateway), audit logging (LLM Gateway + SmithDB), evaluation gates at deploy (LangSmith)
2: Actuating (consequential actions)	Sending communications, writing to systems of record, multi-step workflows, modifying customer-facing data	All Tier 01, plus HITL approval (LangGraph interrupts), scoped tool tokens (Tool Gateway), rate limits, real-time policy enforcement (LLM Gateway)
3: Autonomous / High-stakes (long-horizon, regulated, financial)	Long-horizon autonomous workflows; financial, legal, or medical actions; agent-to-agent coordination	All Tier 02, plus pre-deployment red-team simulation, dedicated runtime guardrails, behavioral anomaly monitoring (Insights Agent), named accountable owner, regulatory mapping, sandboxed code execution

Risk categories: software vs. agents

RISK CATEGORY	TRADITIONAL SOFTWARE	AGENTIC AI	WHERE ENFORCEMENT LIVES
Input manipulation	SQL injection, XSS	Prompt injection, jailbreaking	LLM Gateway input filtering
Data exposure	Direct DB access	Context window leakage, retrieval over-permissioning	LLM Gateway PII / secrets redaction
Authorization	API auth checks	Tool access control, model allowlisting	Tool Gateway, LLM Gateway model allowlists
Auditability	Transaction logs	Full agent decision trace with reasoning	LangSmith + SmithDB
Cost control	Infrastructure spend	Token budgets, model routing, per-agent FinOps	LLM Gateway
Goal alignment	N/A	Goal misspecification, reward hacking, scope creep	Evaluation + HITL (LangGraph interrupts)
Compounding errors	Bounded by retry logic	Errors compound across reasoning chains	Insights Agent + Engine

Every risk category has a named enforcement point in the stack. Governance commitments without enforcement points become principle without practice.

8. FinOps for Agentic Workloads

FinOps for agents is the most universally absent capability in enterprise AI programs. Agent systems consume tokens continuously, call multiple models per interaction, and run in long-lived background processes. Without an explicit operating model for cost, agentic workloads silently destroy unit economics.

FinOps in the Agentic Operating Model is anchored in the LangSmith LLM Gateway (Section 6). Every cost lever and every attribution dimension is enforced there.

The three cost levers

LEVER	WHAT IT CONTROLS	SPECIFIC CONTROLS (ENFORCEMENT POINT)
Routing	Which model handles which node	Auto-route to cheaper models when quality permits; reserve frontier models for high-stakes nodes (LLM Gateway routing policy)
Caching	Whether repeated work re-runs	Eliminate redundant model calls (platform caching layer)
Budgets	What spend is allowed at all	Hard limits per org/workspace/key/user and token budgets per agent/tenant/window; spend alerts on deviation; concurrency caps (runtime quota); token and wall-clock caps on long-running autonomous agents (Deep Agents), all enforced at the LLM Gateway

Cost attribution as the foundation

ATTRIBUTION DIMENSION	WHY IT MATTERS	WHERE THE DATA COMES FROM
Per-agent	Lets product teams own their unit economics	LangSmith trace metadata + Gateway
Per-tenant	Required for chargeback in multi-tenant deployments	Gateway workspace dimension
Per-feature	Identifies which user-facing capabilities pay back	Trace metadata
Per-node	Surfaces which steps in the graph drive cost	LangGraph trace
Per-interaction	Links cost to business outcome (revenue, deflection, NPS)	LangSmith + business event join

The right model is the cheapest model that meets the quality bar, not the most capable one available. Cost-per-correct-answer should be tracked alongside accuracy.

9. Standards Alignment

The Agentic Operating Model is designed to be compatible with the major external guidelines and regulations an enterprise must already answer to. The Risk Taxonomy (Section 7) maps to the risk classification each requires; the LLM Gateway, SmithDB, and audit logging produce the evidence.

STANDARD	WHAT IT REQUIRES	HOW THE AOM SATISFIES IT
NIST AI RMF + 600-1	GOVERN / MAP / MEASURE / MANAGE across the lifecycle, plus 12 GenAI risk categories	The ADLC phases, the risk taxonomy (Sec 7), and Gateway audit logging
EU AI Act	Risk classification, transparency, human oversight for high-risk systems	Risk tiers (Sec 7), HITL interrupts, full trace audit
ISO/IEC 42001	AI management system: policy, planning, operation, evaluation, improvement	The operating model itself, plus the continuous ADLC
AI TRISM (Gartner)	Governance, runtime inspection, information governance, infrastructure security	LLM Gateway, observability, SmithDB

10. Independence Across People, Process, and Technology

Single-vendor agent platforms look fast on day one and expensive on day 365. Agents that run on proprietary runtimes, store prompts in proprietary registries, and emit traces in proprietary formats are agents you cannot move. The Agentic Operating Model treats vendor independence as a design requirement across all three pillars.

Three risks of single-vendor lock-in

RISK	WHAT IT MEANS
Lock-in cost	When pricing changes or a vendor deprecates a feature, you are stuck. Migration cost grows with every prompt, eval, and trace stored in a proprietary system.
Capability ceiling	Your agents can do only what your single vendor supports. Best-in-class models, tools, and orchestration patterns from elsewhere stay out of reach.
Compliance fragility	Regulatory rules change. A vendor that misses a regional deadline takes your compliance posture down with them, and your agents go dark with it.

What keeps each pillar portable

PILLAR	WHAT KEEPS IT INDEPENDENT
People	Open, transferable skills (prompt design, evaluation, agent engineering), not vendor certifications.
Process	The ADLC is a pattern, not a product. Build → Test → Deploy → Monitor → Iterate → Govern runs on any capable stack.
Technology	Open, inspectable harnesses (MIT-licensed LangChain, LangGraph, Deep Agents), open trace formats (OpenTelemetry), open protocols (MCP, A2A, ACP), and a multi-provider gateway you can re-route

Neutrality is offense, not just defense

The best model shifts fast: one 20% better on your workload can land with weeks of notice. Neutrality lets you switch to it immediately, run a cheap model for routing and a frontier model for the high-stakes node, and use cross-provider ensembles and judges. It captures upside from a rapidly improving market, not just insurance against lock-in. LangChain's stack is designed to coexist with whichever model providers, cloud agent platforms, and orchestration ecosystems the enterprise already uses. You should never have to pick one.

Part V · Journey

11. The Maturity Journey

Agent engineering maturity has four stages: Exploring → Building → Operating → Scaling. Most enterprises entering agent development today look like Stage 1 (Exploring): one team prototyping with LLMs, inconsistent tracing, manual testing. The path to Stage 4 (Scaling), where agent engineering is an organizational capability, is structured. It does not happen by accident, and it does not happen as a pure technology upgrade. Every stage transition is a People + Process + Technology move together.

Organizations are usually uneven across dimensions: strong observability with weak evals, or strong evals with little human alignment. The gap between dimensions is the opportunity.

Stage 1 · Exploring: Agents as experiments

DIMENSION	WHAT IT LOOKS LIKE
Summary	One team prototyping with LLMs. Tracing is new or inconsistent.
People	A handful of curious engineers. No defined disciplines; Ring 3 not yet involved.
Process	Ad-hoc pilots. Manual testing. No shared lifecycle.
Technology	Whatever each team picked. Probably notebooks.
Signature signals	<10K monthly traces; 1–2 active projects; <5% engineering adoption; no offline eval runs

Stage 2 · Building: Agents with measurable quality

DIMENSION	WHAT IT LOOKS LIKE
Summary	A team has shipped to production. Basic tracing and offline evals in place for 1–2 apps.
People	A nascent Ring 1 platform team forms; first Ring 2 BU teams co-build with it. First SME (Ring 3) reviewers.
Process	Shared ADLC. Offline evals against first datasets. Developers check traces when something breaks.
Technology	Shared platform: LangChain + LangGraph + LangSmith tracing in production. First Fleet pilots in Ring 3.
Signature signals	10K–500K monthly traces; 3–5 active projects; 5–20% engineering adoption; 1–10 offline eval runs/month

Stage 3 · Operating: Agents with closed-loop learning

DIMENSION	WHAT IT LOOKS LIKE
Summary	Multiple apps in production with systematic observability, online evals, human review, and production intelligence. Quality is actively managed.
People	All three disciplines hired and reskilled to scale; Ring 3 builders active across multiple BUs; annotation queues run regularly.
Process	Online evals (LLM-as-judge on sampled prod traces) plus offline eval suites that gate deploys. FinOps treated alongside latency and accuracy as a core quality dimension. Failure-to-fix cycle time tracked.
Technology	LLM Gateway enforcing budgets and policy. SmithDB powering fast trace search and FinOps attribution. Sandboxes for actuating tiers.
Signature signals	500K–10M monthly traces; 6–15 active projects; 20–50% engineering adoption; 11–100 offline eval runs/month; online eval rules running on prod

Stage 4 · Scaling: Agents as an organizational capability

DIMENSION	WHAT IT LOOKS LIKE
Summary	Agent engineering is an organizational capability. Multiple teams, automated improvement loops, cost governance, proactive insights.
People	Universal AI fluency. Platform team serves the org; non-engineers (PMs, SMEs) also on the platform. Agent building is a normal job activity, not a specialist function.
Process	Improvement loop runs continuously. Governance is proactive, not reactive. Online eval results auto-feed offline datasets. Annotation focused on novel / edge cases.
Technology	Insights Agent generating clusters from production traces (production intelligence at scale). LangSmith Engine running the auto-loop end to end (detect → diagnose → fix → eval). SmithDB making the trace corpus fast enough to close the loop in seconds.
Signature signals	10M+ monthly traces; 15+ active projects; 50%+ engineering adoption; 100+ offline eval runs/month; continuous online eval as the primary quality signal

What each transition requires

TRANSITION	KEY UNLOCK	PEOPLE MOVE	PROCESS MOVE	TECHNOLOGY MOVE
Exploring → Building	A hanGet traces flowing from production and run your first evalsdful of curious	Form Ring 1; identify Ring 2 first-wave teams; first Ring 3 pilots	Adopt a shared ADLC; create the first dataset; run the first offline experiment	Enable LangSmith tracing in prod; LangChain/ LangGraph as the default build layer
Building → Operating	Move from offline-only to online evals; close the loop between prod data and development	Hire to anchor each discipline; reskill broadly; expand Ring 3 enablement	LLM-as-judge rules on prod traces; annotation queues; calibrate evaluators against human labels; treat FinOps as a quality dimension	LLM Gateway, SmithDB, Sandboxes; Fleet at scale; quality dashboards
Operating → Scaling	Stop drowning in traces; let the platform surface what matters	Universal AI fluency across the workforce; agent building is a normal job activity	Improvement loop runs continuously; governance is proactive; online eval results auto-populate offline datasets	Insights for clustering and trend analysis; LangSmith Engine running the auto-loop; cost attribution and automated remediation

How LangChain supports each transition

Moving through these stages is rarely a pure technology problem. LangChain provides support against all three pillars.

FUNCTION	WHAT IT COVERS	MOST RELEVANT STAGE
Deployed Engineering	Embedded engineering advisory for high-priority initiatives. Architectural guidance and best-practice transfer.	Exploring → Building, and Building → Operating
Enablement & Education	Structured curriculum for upskilling existing engineers into agent engineers, platform engineers, and evaluation engineers. Workshops, certifications, reference implementations. Includes the AI literacy tracks that scale Ring 3 adoption.	Every stage; heaviest Building → Operating → Scaling
Professional Services	Engagement-based delivery focused on agent engineering. Co-builds production agents with the customer's team, instilling eval-driven iteration cycles, prompt versioning discipline, and trace-based debugging as habits that outlast the engagement.	Building → Operating, and Operating → Scaling
Support	Production-grade support for LangSmith, the LLM Gateway, and the deployment runtime. Incident response, troubleshooting, escalation paths.	Every stage; criticality grows with maturity
Account Management	Strategic relationship management across the customer's agent portfolio. Roadmap alignment, contract evolution, adoption tracking.	Every stage

The maturity journey is one organization's responsibility to walk. LangChain's role is to make sure that journey is faster, less risky, and less lonely.

12. In Practice

Below, we describe two organizations operating at very different scales, each illustrating what happens when People, Process, and Technology line up around the same goals. The first prioritized velocity without giving up centralization; the second prioritized governance across a heavily regulated footprint. Both grew into platforms that scale without fragmenting.

A global automotive manufacturer

This organization's Enterprise AI team started with a clear mandate and a hard constraint: get AI into production across supply chain, manufacturing, R&D, and customer-facing properties, without creating a centralized bottleneck that slowed every business unit down.

Working with LangChain, they built a central platform on LangGraph and LangSmith that functions as the foundation every agent team builds on. The results reflect what happens when the platform and org structure are set up correctly. Agent deployment time dropped from three months to one week. One use case that previously took six months to reach production shipped in four days. A single engineer can now spin up a new production agent end-to-end.

The platform now runs approximately 50 agents in production across the organization, with 300,000 traces per week flowing through LangSmith. The team issued an enterprise-wide mandate: every production AI application, regardless of framework, pushes telemetry to LangSmith, driving a single observability standard across LangGraph, CrewAI, LlamaIndex, OpenAI Agent Builder, and AgentCore. It replaced fragmented monitoring with a consistent signal the central team can act on.

The business impact is concrete. A manufacturing agent saves millions of dollars by rapidly diagnosing line stoppages. An AI intake bot replaced a sixty-to-seventy question manual process previously managed by an external firm. Their internal multi-agent platform serves 20,000 daily active users, with an agent builder tool targeting rollout to 56,000 employees.

What made this work wasn't any single tool. It was the combination of a governed platform, a central team that operated as a hub rather than a bottleneck, and a reskilling model that turned high-performing engineers into agent builders without replacing them.

A global telecommunications provider

This organization's challenge was different. Operating across dozens of countries and business entities, their priority wasn't velocity. It was governance at scale. Different regulatory environments, strict data isolation requirements between entities, and thousands of employees who needed to access AI capabilities without creating uncontrolled sprawl.

Working with LangChain, they built a production platform for enterprise AI on LangChain, LangGraph, and LangSmith, now serving over 100 enterprise customers. Internally, they are rolling out self-hosted LangSmith instances across multiple entities, with governance, PII filtering, and access isolation built into the foundation rather than retrofitted later.

The architectural decisions they made early are exactly the ones that will let them scale to thousands of employees without rebuilding the governance layer when requirements change: open harnesses, self-hosted observability, ABAC controls defined at the platform level from the start.

13. Critical Success Factors

FACTOR	PILLAR	WHY IT MATTERS
Build evaluation infrastructure	Process + Tech	Without it, you have no feedback loop. Golden datasets before first deployment.
Centralize platform, decentralize delivery, distribute fluency	People	All three failure modes (centralized bottleneck, decentralized sprawl, engineer-only) are equally common.
Treat prompts as production	Process + Tech	Version-control every prompt. Measure the impact of every change.
Monitor everything from day one	Tech	Observability added retroactively is orders of magnitude harder.
Embed security and compliance in the	Tech (+ People + Process)	Late reviews create bottlenecks and miss architectural risks. The LLM Gateway makes enforcement automatic.
Treat FinOps as a quality dimension	Process + Tech	Agentic costs scale super-linearly. Token budgets and per-agent attribution from day one.
Define ownership per agent	People	Without it, quality degrades with no accountable party. Every agent has a named owner across all three rings.
Invest in Ring 3 fluency	People	The largest fraction of the workforce sits here. Treating AI fluency as a specialist skill is the most predictable way to be overtaken.
Plan for iteration, not perfection	Process	The first version of any agent will be wrong. Budget cycles. Measure improvement.

Conclusion

The shift to agents is an organizational transformation, rather than a technology upgrade.

The technology is here today. LangChain, LangGraph, and Deep Agents provide the OSS build layer at every abstraction level. LangSmith, together with Engine, Chat, and Insights, provides observability, evaluation, and the closed-loop improvement engine. LangSmith Deployment and Sandboxes provide the runtime and isolation. SmithDB makes the trace and evaluation corpus fast enough to query continuously. The LangSmith LLM Gateway anchors governance, security, and FinOps in a single enforcement point.

What remains is the organizational will to adopt a new way of building software, where:

- Evaluation precedes deployment,
- Traces precede debugging,
- Production data precedes the next iteration, and
- Every employee, not only engineers, participates in agent creation and improvement.

The Agentic Operating Model is how an enterprise puts that into practice across People, Process, and Technology, together.

[Speak with the LangChain team](#) to learn more about accelerating your agent development initiatives.